

Python E-Course

Module 3 — Control Flow

Detailed Notes

Module Overview

Level: Beginner

Duration: ~1 week (6-8 hours)

Prerequisites: Modules 1-2

What You'll Learn

- Make decisions with if, elif, else.
- Use while and for loops to repeat work.
- Control loops with break, continue, and else.
- Iterate over numeric ranges with range().
- Use Python 3.10+'s match/case for structural patterns.

Lessons

#	Lesson	Duration
1	if / elif / else	30 min
2	while Loops	30 min
3	for Loops and range()	30 min
4	break, continue, else on loops	25 min
5	Nested Conditions and Loops	30 min
6	match / case (Python 3.10+)	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 4: Functions — bundle code into reusable, named units.

Lesson 1: if / elif / else

Module: 3 — Control Flow

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Write if, if/else, and if/elif/else statements.
- Use comparison and logical operators in conditions.
- Use one-line conditional expressions (ternary).

Prerequisites

- Module 2 — All lessons

1. Why This Matters

Decisions are the soul of programming. Until now, your code runs line by line, top to bottom. if lets it choose — and that's what turns scripts into actual programs.

2. Concept

2.1 if

```
if condition:
    # runs when condition is truthy
    ...
```

The condition is any expression that evaluates to truthy or falsy. The indented block runs only if it's truthy.

```
temperature = 30
if temperature > 25:
    print("It's hot.")
```

2.2 if/else

```
if score >= 60:
    print("Pass")
else:
    print("Fail")
```

Exactly one branch runs.

2.3 if/elif/else

Chain conditions:

```
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
```

```

elif score >= 60:
    grade = "C"
else:
    grade = "F"

```

Conditions are tested top-to-bottom; the first match wins. else (optional) catches everything else.

2.4 Nested if

You can put if inside if:

```

if logged_in:
    if is_admin:
        print("admin panel")
    else:
        print("user dashboard")

```

Prefer flattening with and/or when readable:

```

if logged_in and is_admin:
    print("admin panel")

```

2.5 Ternary expression

A one-line if/else for assigning a value:

```

status = "adult" if age >= 18 else "minor"

```

Reads "status is adult if age >= 18 else minor." Use sparingly — only when it improves clarity.

2.6 Truthiness in conditions

Any value works as a condition; empty containers, 0, "", and None are falsy.

```

name = ""
if name:
    print("Hi,", name)
else:
    print("No name given")

```

3. Code Examples

Example 1: Pass/fail

```

score = 72
if score >= 60:
    print("Pass")
else:
    print("Fail")

```

Expected Output: Pass

Explanation: Single condition; only the matching branch runs.

Example 2: Grade band

```
score = 83
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
elif score >= 60:
    grade = "C"
else:
    grade = "F"
print(f"Grade: {grade}")
```

Expected Output: Grade: B

Explanation: First condition that's True wins.

Example 3: Combined conditions

```
age = 25
has_license = True
if age >= 18 and has_license:
    print("Can drive")
else:
    print("Cannot drive")
```

Expected Output: Can drive

Explanation: Both clauses must be true for and.

Example 4: Ternary

```
age = 16
status = "adult" if age >= 18 else "minor"
print(status)
```

Expected Output: minor

Explanation: One-line conditional.

Example 5: Truthiness

```
items = []
if items:
    print("non-empty")
else:
    print("empty list")
```

Expected Output: empty list

Explanation: Empty list is falsy — idiomatic to test directly, not if len(items) == 0.

4. Common Mistakes

Mistake	What Happens	Fix
= instead of == in condition	SyntaxError	Use ==

Forgetting :	SyntaxError	Add colon
Mixing tabs and spaces in block	IndentationError	Use 4 spaces
if x == True:	Works but verbose	if x:

5. Practice Tasks

- Ask for a number; print even or odd.
- Map a score (0-100) to A/B/C/D/F using if/elif/else.
- Print "weekend" if a hard-coded day (e.g., "Sat") is Sat or Sun, else "weekday".

6. Key Takeaways

- if, elif, else choose one branch.
- Conditions can be any expression; truthiness applies.
- Ternary gives a one-line value choice.
- Flatten nested ifs with and/or where it reads cleaner.

7. What's Next

Next: while loops — repeat actions until a condition fails.

Lesson 2: while Loops

Module: 3 — Control Flow

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Write while loops that run until a condition becomes false.
- Avoid infinite loops by updating the loop variable.
- Use sentinel-driven loops to process user input.

Prerequisites

- Module 3 Lesson 1

1. Why This Matters

Loops let your program do the same thing many times — totaling sales, prompting until valid input, polling a server. while is the most general loop: it runs until you tell it to stop.

2. Concept

2.1 Syntax

```
while condition:
    # body – runs as long as condition is truthy
    ...

count = 0
while count < 5:
    print(count)
    count += 1
```

Pattern: initialize a control variable, test it in the condition, update it inside the body.

2.2 Infinite loops

If you forget to update the variable, the loop runs forever:

```
count = 0
while count < 5:
    print(count)    # MISSING count += 1 – infinite!
```

Stop a runaway with Ctrl+C. Then fix the update.

2.3 Sentinel-driven loops

Run until a special "sentinel" value appears — common for user input:

```
answer = ""
while answer != "quit":
```

```
answer = input("Type something (or 'quit'): ")
print("You said:", answer)
```

2.4 Counter and accumulator patterns

```
total = 0
n = 1
while n <= 100:
    total += n
    n += 1
print(total)    # 5050
```

You'll see this pattern constantly — and for loops (next lesson) make it shorter.

2.5 while True + break

Sometimes the exit condition lives inside the loop body. Use while True and break (next lesson) to exit:

```
while True:
    cmd = input("> ")
    if cmd == "exit":
        break
    print("Got:", cmd)
```

3. Code Examples

Example 1: Count down

```
n = 5
while n > 0:
    print(n)
    n -= 1
print("Liftoff!")
```

Expected Output:

```
5
4
3
2
1
Liftoff!
```

Explanation: Decrement each pass; loop ends when n reaches 0.

Example 2: Sum 1 to 10

```
total = 0
i = 1
while i <= 10:
    total += i
    i += 1
print("Sum:", total)
```

Expected Output: Sum: 55

Explanation: Classic accumulator pattern.

Example 3: Sentinel input

```
total = 0
while True:
    line = input("Number (or 'q' to quit): ")
    if line == "q":
        break
    total += int(line)
print("Total:", total)
```

Sample interaction:

```
Number (or 'q' to quit): 10
Number (or 'q' to quit): 20
Number (or 'q' to quit): 30
Number (or 'q' to quit): q
Total: 60
```

Explanation: Loop forever, exit when sentinel typed.

Example 4: Validate input

```
age = -1
while age < 0:
    age = int(input("Age: "))
print(f"Age accepted: {age}")
```

Sample interaction:

```
Age: -5
Age: 25
Age accepted: 25
```

Explanation: Re-prompt until non-negative.

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting to update counter	Infinite loop	Add count += 1
Off-by-one: while i < 5 prints 0-4, not 1-5	Wrong count	Use <= 5 or start from 1
Updating before testing	Body skipped or runs extra	Match increment placement to intent
Sentinel typed in wrong type	int("q") raises	Test sentinel before converting

5. Practice Tasks

- Print numbers 10 down to 1 using while.
- Sum even numbers from 1 to 50 with while.
- Keep asking for a password until the user types "open sesame". Print Welcome.

6. Key Takeaways

- `while condition`: repeats while the condition is truthy.
- Always have a way for the condition to become false (or use `break`).
- Use sentinel values or `while True + break` for input-driven loops.

7. What's Next

`for` loops — Python's idiomatic way to iterate over a range or collection.

Lesson 3: for Loops and range()

Module: 3 — Control Flow

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Use for to iterate over sequences and ranges.
- Use range(start, stop, step) to generate numeric sequences.
- Iterate over strings character by character.
- Use enumerate() to get index + value while iterating.

Prerequisites

- Module 3 Lesson 2

1. Why This Matters

for is Python's go-to loop. It's shorter than while, less error-prone (no counter to forget), and it's how you'll loop over lists, files, API responses, and everything else. Master the for + range pattern first; everything else builds on it.

2. Concept

2.1 Basic for

```
for item in iterable:
    # use item
    ...

for letter in "Python":
    print(letter)
```

item takes each value from iterable once.

2.2 range(stop)

range(n) produces numbers 0, 1, ..., n-1:

```
for i in range(5):
    print(i)          # 0 1 2 3 4
```

2.3 range(start, stop)

range(2, 6) → 2, 3, 4, 5 (start inclusive, stop exclusive):

```
for i in range(1, 11):
    print(i)          # 1..10
```

2.4 range(start, stop, step)

```
for i in range(0, 20, 5):
    print(i)          # 0 5 10 15
```

```
for i in range(10, 0, -1):
    print(i)          # 10..1
```

2.5 Iterating over a string

A string is iterable — each character in turn:

```
for ch in "hi":
    print(ch)          # h, then i
```

2.6 enumerate(iterable)

Get both index and value:

```
for i, letter in enumerate("abc"):
    print(i, letter)
# 0 a
# 1 b
# 2 c
```

Optional start index: `enumerate("abc", start=1)` starts at 1.

2.7 When to use which loop

Use	When
for	You know what you're iterating over (range, string, list, file)
while	The exit condition is dynamic and not tied to a sequence

3. Code Examples

Example 1: Count 1-5

```
for i in range(1, 6):
    print(i)
```

Expected Output:

```
1
2
3
4
5
```

Explanation: `range(1, 6)` → 1, 2, 3, 4, 5.

Example 2: Sum 1 to 100

```
total = 0
for n in range(1, 101):
    total += n
print(total)
```

Expected Output: 5050

Explanation: Same as the while-loop example, but shorter.

Example 3: Step

```
for n in range(0, 21, 5):  
    print(n)
```

Expected Output:

```
0  
5  
10  
15  
20
```

Explanation: Step of 5 from 0 up to (not including) 21.

Example 4: Reverse

```
for n in range(5, 0, -1):  
    print(n)
```

Expected Output:

```
5  
4  
3  
2  
1
```

Explanation: Negative step counts down.

Example 5: enumerate

```
items = ["apple", "banana", "cherry"]  
for index, item in enumerate(items, start=1):  
    print(f"{index}. {item}")
```

Expected Output:

```
1. apple  
2. banana  
3. cherry
```

Explanation: start=1 for human-friendly numbering. Lists come formally in Module 5 — for now, treat this as an iterable.

Example 6: Character count

```
word = "hello"  
count = 0  
for ch in word:  
    count += 1  
print(count)
```

Expected Output: 5

Explanation: Iterating a string yields characters; this is just `len(word)` — but illustrates for over a string.

4. Common Mistakes

Mistake	What Happens	Fix
<code>range(1, 10)</code> expecting 10 to be included	Stops at 9	Use <code>range(1, 11)</code>
Forgetting <code>range()</code> is exclusive of stop	Off-by-one	Always think: "stop is one past the end"
Calling <code>len(range(...))</code> to count	Works but pointless	The range itself tells you the size
Modifying loop variable inside body	Confusing behavior	Don't reassign <code>i</code> inside a <code>for i in range(...)</code>

5. Practice Tasks

- Print all even numbers from 2 to 20 using `for` + `range`.
- Compute the product of numbers 1 to 5 (factorial) with `for`.
- Print each character of "PYTHON" on its own line with its 1-based index.

6. Key Takeaways

- `for x in iterable:` is Python's main loop.
- `range(start, stop, step)` — stop is exclusive.
- Iterating a string yields characters.
- `enumerate()` gives you index + value.

7. What's Next

We'll learn how to change the flow inside a loop with `break`, `continue`, and the `loop else` clause.

Lesson 4: break, continue, and the loop else

Module: 3 — Control Flow

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Use break to exit a loop early.
- Use continue to skip to the next iteration.
- Understand the rarely-used else clause on loops.

Prerequisites

- Module 3 Lesson 3

1. Why This Matters

Real loops are rarely "run from 1 to 10 doing the same thing every time." You'll need to stop early when you find what you're looking for, skip bad data, or detect "we never found a match." break, continue, and loop-else cover all three.

2. Concept

2.1 break

Exits the innermost loop immediately. Control jumps to the line after the loop.

```
for n in range(10):
    if n == 4:
        break
    print(n)
# 0 1 2 3
```

2.2 continue

Skips the rest of the current iteration. Loop continues with the next value.

```
for n in range(5):
    if n % 2 == 0:
        continue
    print(n)
# 1 3
```

2.3 else on a loop

The else block runs only if the loop completes without break. Useful for "didn't find it" cases:

```
for n in [1, 2, 3]:
    if n == 5:
        print("Found!")
        break
```

```
else:
    print("Not found.")
```

If you break, else is skipped. If you don't, else runs.

2.4 Choosing between them

Situation	Use
Found what I was looking for — stop	break
Bad item — skip and move on	continue
"If we got through without finding it"	loop else

3. Code Examples

Example 1: Find the first multiple of 7

```
for n in range(1, 100):
    if n % 7 == 0:
        print("First:", n)
        break
```

Expected Output: First: 7

Explanation: Loop exits the moment we find a hit.

Example 2: Skip negatives

```
nums = [3, -1, 5, -2, 7]
total = 0
for n in nums:
    if n < 0:
        continue
    total += n
print(total)
```

Expected Output: 15

Explanation: Skips negatives; adds positives. $3 + 5 + 7 = 15$.

Example 3: Loop else — searching

```
target = 8
for n in [1, 2, 3, 4, 5]:
    if n == target:
        print("Found")
        break
else:
    print("Not found")
```

Expected Output: Not found

Explanation: Loop ran fully without break, so else ran.

Example 4: Validate password attempts

```
correct = "py123"
attempts = 3
while attempts > 0:
    pw = input("Password: ")
    if pw == correct:
        print("Welcome!")
        break
    attempts -= 1
    print(f"Wrong. {attempts} attempts left.")
else:
    print("Locked out.")
```

Sample:

```
Password: x
Wrong. 2 attempts left.
Password: y
Wrong. 1 attempts left.
Password: z
Wrong. 0 attempts left.
Locked out.
```

Explanation: Loop else runs when the while condition fails normally (attempts hit 0). If the password matches, break skips the else.

4. Common Mistakes

Mistake	What Happens	Fix
break thinking it exits all loops	Only exits innermost	Use flag or refactor to function (Module 4)
continue skipping the counter update in while	Infinite loop	Update counter before continue
Misusing loop else thinking it's "the other branch"	Runs every time you don't break	Use only for "search failed" semantics

5. Practice Tasks

- Print numbers 1-20, skipping multiples of 3.
- Find the first number > 100 in [1, 50, 80, 120, 200] using break.
- Search for 9 in range(1, 6) using loop else to print "missing" when not found.

6. Key Takeaways

- break exits the loop entirely.
- continue jumps to the next iteration.
- Loop else runs only when the loop completes without break — great for "didn't find" cases.

7. What's Next

Nested conditions and loops — combining what we have to solve richer problems.

Lesson 5: Nested Conditions and Loops

Module: 3 — Control Flow

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Combine nested if statements.
- Write nested loops to generate grids, pairs, and tables.
- Use early returns / breaks to flatten deep nesting.

Prerequisites

- Module 3 Lesson 4

1. Why This Matters

Real logic isn't always one decision deep. A login check might be "is the user known and are they active and is their password correct?" A multiplication table needs a row loop and a column loop. Nesting is unavoidable — but deep nesting is a smell. This lesson teaches both how to nest and when to refuse.

2. Concept

2.1 Nested if

An if inside another if:

```
if logged_in:
    if is_admin:
        print("admin")
    else:
        print("user")
else:
    print("guest")
```

Often you can flatten with and/or:

```
if logged_in and is_admin:
    print("admin")
elif logged_in:
    print("user")
else:
    print("guest")
```

2.2 Nested loops

A loop inside another loop. The inner loop runs fully for each iteration of the outer:

```
for row in range(3):
    for col in range(3):
```

```

        print(f"({row},{col})", end=" ")
    print()

```

Output:

```

(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)

```

2.3 Cost of nesting

A loop of N inside a loop of M runs $N \times M$ times. Two nested loops over 1000 items each → 1,000,000 iterations. Watch out for this in Module 5.

2.4 Flattening tricks

- Guard clauses: handle the "bad" case early.

```

`python if not logged_in: print("not allowed") else: # do work `

```

- continue in loops to skip rather than nest.
- Functions (Module 4) — pull the inner logic out.

3. Code Examples

Example 1: Nested if — login check

```

logged_in = True
is_admin = False
if logged_in:
    if is_admin:
        print("Admin panel")
    else:
        print("User dashboard")
else:
    print("Please log in")

```

Expected Output: User dashboard

Explanation: Two checks; the path is "logged in but not admin".

Example 2: Multiplication table

```

for row in range(1, 6):
    for col in range(1, 6):
        print(f"{row*col:4d}", end=" ")
    print()

```

Expected Output:

```

1  2  3  4  5
2  4  6  8 10
3  6  9 12 15
4  8 12 16 20
5 10 15 20 25

```

Explanation: Outer loop: rows; inner: columns. :4d pads each number.

Example 3: Find a pair that sums to 10

```
nums = [1, 3, 5, 7, 2, 8]
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == 10:
            print(nums[i], "+", nums[j])
```

Expected Output:

```
3 + 7
2 + 8
```

Explanation: Inner loop starts at i+1 to avoid using the same index twice. Lists formally in Module 5.

Example 4: Guard-clause flatten

```
# Nested version
def show_dashboard(user):
    if user is not None:
        if user["active"]:
            print("welcome")
        else:
            print("inactive")
    else:
        print("no user")

# Flatter version
def show_dashboard2(user):
    if user is None:
        print("no user")
        return
    if not user["active"]:
        print("inactive")
        return
    print("welcome")
```

Explanation: The second reads top-to-bottom: "handle bad cases first, then the happy path." (def and return formally in Module 4.)

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting inner print() after inner loop	All cells on one line	Add print() after inner loop to start a new row
Two loops using the same variable name	Shadows; confusing	Use distinct names: i and j, or row and col
Three+ levels of nesting	Hard to read	Extract into a function (Module 4)
Slow due to nested loops on	Performance	Use sets/dicts (Module 5) for

large data		O(1) lookups
------------	--	--------------

5. Practice Tasks

- Print a 5×5 grid of * characters.
- Print a multiplication table for 1-10.
- Given nums = [4, 1, 7, 9, 2], find and print all pairs whose sum is odd.

6. Key Takeaways

- Nest when needed; flatten when possible.
- Inner loop runs fully per outer iteration — watch for N×M cost.
- Guard clauses keep nesting shallow.

7. What's Next

The final lesson — match/case, Python 3.10's structural pattern matching.

Lesson 6: match / case (Python 3.10+)

Module: 3 — Control Flow

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Use match/case for value-based dispatch.
- Use | to combine patterns.
- Use _ as the wildcard (default) case.

Prerequisites

- Module 3 Lesson 5
- Python 3.10 or newer.

1. Why This Matters

When you have a chain of if/elif/elif/else checking what a single value equals, match/case reads more like a table — easier to scan and harder to mess up. It's also the foundation for structural pattern matching, which we'll fully use in Module 9 (OOP) and Module 12 (JSON shapes).

2. Concept

2.1 Basic syntax

```
match value:
    case pattern_1:
        ...
    case pattern_2:
        ...
    case _:
        ... # default
```

_ (underscore) is the catch-all "wildcard" — always matches.

2.2 Literal patterns

```
match command:
    case "start":
        print("Starting...")
    case "stop":
        print("Stopping...")
    case "pause":
        print("Pausing...")
    case _:
        print("Unknown command")
```

2.3 Combining patterns with |

```
match day:
    case "Sat" | "Sun":
```

```

        print("Weekend")
    case "Mon" | "Tue" | "Wed" | "Thu" | "Fri":
        print("Weekday")
    case _:
        print("Unknown day")

```

2.4 Capture patterns

A bare name captures the value into a variable:

```

match status_code:
    case 200:
        print("OK")
    case 404:
        print("Not found")
    case code:
        print(f"Got status {code}")

```

The last case captures any unmatched value into code.

2.5 When to choose match vs if/elif

- Dispatching on value or shape of a single thing → match.
- Boolean expressions across multiple variables → if/elif.

3. Code Examples

Example 1: Command dispatch

```

command = "start"
match command:
    case "start":
        print("Starting...")
    case "stop":
        print("Stopping...")
    case _:
        print("Unknown")

```

Expected Output: Starting...

Explanation: Matches the first literal pattern.

Example 2: Day of week

```

day = "Sun"
match day:
    case "Sat" | "Sun":
        print("Weekend")
    case _:
        print("Weekday")

```

Expected Output: Weekend

Explanation: | allows multiple matching values per case.

Example 3: Capture

```
status = 503
match status:
    case 200:
        print("OK")
    case 404:
        print("Not found")
    case code:
        print(f"Other status: {code}")
```

Expected Output: Other status: 503

Explanation: No literal matched, so the bare name code captured the value.

Example 4: Compare with if/elif

```
# match version
def http_message(code):
    match code:
        case 200:
            return "OK"
        case 301 | 302:
            return "Redirect"
        case 404:
            return "Not Found"
        case _:
            return "Other"

print(http_message(302))
```

Expected Output: Redirect

Explanation: Reads as a small table. (Functions formally in Module 4 — included for context.)

4. Common Mistakes

Mistake	What Happens	Fix
Running on Python 3.9 or below	SyntaxError	Upgrade to 3.10+
Using match for complex boolean expressions	Doesn't fit	Use if/elif
Confusing case x: (capture) with literal	Always matches; later cases unreachable	Use literal case 5: not case 5 — wait, both work. The issue is bare names: case foo: captures into foo. Use literals or case _: for default
Forgetting _ default	Unmatched value silently does nothing	Add case _:

5. Practice Tasks

- Write a match for a variable light taking values "red", "yellow", "green" and printing "stop", "slow", "go".
- Map HTTP codes 200/301/404/500 to messages using match.
- Print whether a month string is Spring (Mar/Apr/May), Summer, Autumn, Winter using | combinations.

6. Key Takeaways

- match is a clearer alternative to long if/elif chains keying off one value.
- | combines patterns; _ is the default.
- Bare names in a case capture, not literal-compare.
- Requires Python 3.10+.

7. What's Next

That wraps Module 3. Up next: Module 4 — Functions.

Module 3 — Exercises

18 exercises (3 per lesson).

Lesson 3.1 — if / elif / else

Exercise 1: Pass/Fail (Easy)

Task: Given score = 55, print Pass if ≥ 40 else Fail.

Expected Output: Fail

Exercise 2: Grade (Medium)

Task: Ask for a score 0-100 and print A/B/C/D/F.

Sample: Score: 73 $\rightarrow$ Grade: C

Exercise 3: Leap year (Hard)

Task: Ask for a year. Print whether it's a leap year. Rule: divisible by 4, except centuries not divisible by 400.

Sample: 2024 $\rightarrow$ Leap year, 2100 $\rightarrow$ Not a leap year

Lesson 3.2 — while

Exercise 1: Countdown (Easy)

Task: Print 10 down to 1 using while.

Exercise 2: Sum until > 100 (Medium)

Task: Starting at 1, keep adding consecutive integers until the running total exceeds 100. Print the final total and the last number added.

Exercise 3: Number guessing (Hard)

Task: Hard-code secret = 42. Repeatedly ask the user for guesses; print higher/lower/correct. Stop on correct guess.

Lesson 3.3 — for and range

Exercise 1: Even numbers (Easy)

Task: Print even numbers from 2 to 20 using for + range.

Exercise 2: Factorial (Medium)

Task: Compute 6! using for. Expected output: 720.

Exercise 3: Sum of squares (Hard)

Task: Compute the sum of squares from 1^2 to 10^2 . Expected output: 385.

Lesson 3.4 — break / continue / else

Exercise 1: Skip 5s (Easy)

Task: Print numbers 1 to 10, skipping any divisible by 5, using continue.

Exercise 2: First > 50 (Medium)

Task: Given nums = [3, 17, 42, 55, 8, 60], print the first number greater than 50 using break.

Exercise 3: Prime check (Hard)

Task: Ask for n; use a for loop with else to print prime or not prime. Hint: check divisors 2 to $\sqrt{n}$.

Lesson 3.5 — Nested

Exercise 1: Square grid (Easy)

Task: Print a 6×6 grid of * characters.

Exercise 2: Multiplication table (Medium)

Task: Print the multiplication table for 1-7.

Exercise 3: Triangle (Hard)

Task: Print a right triangle of #, 7 rows tall:

```
#
##
###
####
#####
#####
#####
```

Lesson 3.6 — match / case

Exercise 1: Light (Easy)

Task: light = "yellow". Use match to print stop/slow/go.

Exercise 2: HTTP messages (Medium)

Task: Map 200, 301, 404, 500 to short messages using match with | where helpful.

Exercise 3: Season (Hard)

Task: Map a month string to its season using match with |. e.g., Dec/Jan/Feb → Winter.

Module 3 — Quiz

10 multiple-choice questions covering Lessons 1-6.

Question 1

What does this print?

```
x = 10
if x > 5:
    print("big")
elif x > 0:
    print("small positive")
```

A. big B. small positive C. both D. nothing

Answer: A — only the first matching branch runs. (L1)

Question 2

How many times does this print "hi"?

```
for i in range(3):
    print("hi")
```

A. 2 B. 3 C. 4 D. infinite

Answer: B — range(3) is 0, 1, 2. (L3)

Question 3

What does range(2, 10, 3) produce?

A. 2, 5, 8 B. 2, 5, 8, 11 C. 2, 4, 6, 8 D. 3, 6, 9

Answer: A — start 2, step 3, stop exclusive at 10. (L3)

Question 4

What does continue do?

A. Exits the loop B. Skips to the next iteration C. Restarts the loop D. Pauses execution

Answer: B (L4)

Question 5

When does the loop else run?

A. Always after the loop B. Only when the loop body never runs C. Only when the loop completes without break D. Only when there's an exception

Answer: C (L4)

Question 6

What is the output?

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

A. 6 lines B. 3 lines C. 5 lines D. 2 lines

Answer: A — $3 \times 2 = 6$. (L5)

Question 7

Which match pattern matches any value?

A. case all: B. case _: C. case *: D. case any:

Answer: B (L6)

Question 8

What's missing here?

```
count = 0
while count < 5:
    print(count)
```

A. Nothing — this works B. count += 1 inside the loop C. A for loop instead D. An else clause

Answer: B — without it, the loop is infinite. (L2)

Question 9

What does this print?

```
nums = [3, -2, 5, -1, 4]
total = 0
for n in nums:
    if n < 0:
        continue
    total += n
print(total)
```

A. 9 B. 12 C. 14 D. 4

Answer: B — adds $3 + 5 + 4 = 12$. (L4)

Question 10

Which is the simpler / preferred replacement for this code in Python 3.10+?

```
if cmd == "start":
    ...
elif cmd == "stop":
```

```
    ...  
elif cmd == "pause":  
    ...  
else:  
    ...
```

A. A while loop B. match/case C. Ternary expressions D. Nested ifs

Answer: B (L6)

Module 3 — Assignment: Number Guessing Game

Estimated time: 1-1.5 hours

Combines concepts from: Module 3 + Modules 1-2

Brief

Build a CLI number-guessing game that picks a "random" target (hard-coded for now; we'll use random in Module 10) and prompts the user until they guess it. Use while, if/elif/else, and break/continue.

Requirements

- Hard-code target = 73 at the top (any value 1-100).
- Prompt the user for a guess (1-100) repeatedly.
- For each guess, print:
 - Too low if guess < target
 - Too high if guess > target
 - Correct! Took N attempts. and stop on match.
- Reject invalid inputs (out of range 1-100) with a message and continue.
- Limit to 7 attempts. After 7, print Out of attempts. The number was 73. and stop.

Constraints

- Only Modules 1-3 concepts.
- No functions, no lists/dicts.
- Use a while loop and if/elif/else.

Expected Behavior

```
Guess a number 1-100 (you have 7 tries): 50
Too low.
Guess: 75
Too high.
Guess: 70
Too low.
Guess: 73
Correct! Took 4 attempts.
```

Or:

```
Guess: 200
Out of range. Please pick 1-100.
Guess: 60
Too low.
... (7 attempts pass)
Out of attempts. The number was 73.
```

Hints

- Wrap `int(input(...))` to convert the guess.

- Use a counter attempts initialized to 0; attempts += 1 inside the loop before the comparison (only for valid guesses).
- Use break on a correct guess.

Grading Rubric

Criterion	Weight
Correct game logic	35%
Input validation	20%
Attempt limit handled	20%
Clear output	15%
PEP 8 style	10%

Submission

Save as assignment_module3.py.

Module 3 — Mini Project: Multiplication Table Generator

Overview

Build a small CLI tool that prints a formatted multiplication table for any range the user picks. Practices for, range, nested loops, and f-strings.

Concepts Used

- for and range (M3 L3)
- Nested loops (M3 L5)
- f-string formatting (M2 L6)
- input() and int() (M2 L4, L6)

Features

- Ask for start and end numbers.
- Print a multiplication table from start to end (inclusive in both dimensions).
- Each cell right-aligned to width 5.

Starter Code

```
# Mini Project: Multiplication Table Generator

start = int(input("Start: "))
end = int(input("End: "))

# Header row
print("    ", end="")
for c in range(start, end + 1):
    print(f"{c:5d}", end="")
print()
print("    " + "-" * (5 * (end - start + 1)))

# Body
# TODO: print each row with the row label, then each product cell.
```

Expected Interaction

```
Start: 1
End: 5

    1    2    3    4    5
-----
1 |    1    2    3    4    5
2 |    2    4    6    8   10
3 |    3    6    9   12   15
4 |    4    8   12   16   20
5 |    5   10   15   20   25
```

Extension Challenges

- Highlight perfect squares with a * next to the number.
- Add a "skip multiples of N" option.

- Compute the sum of each row and print it as a final column.

Grading Rubric

Criterion	Weight
Correct table	40%
Alignment / formatting	25%
Handles different sizes	20%
Style & names	15%